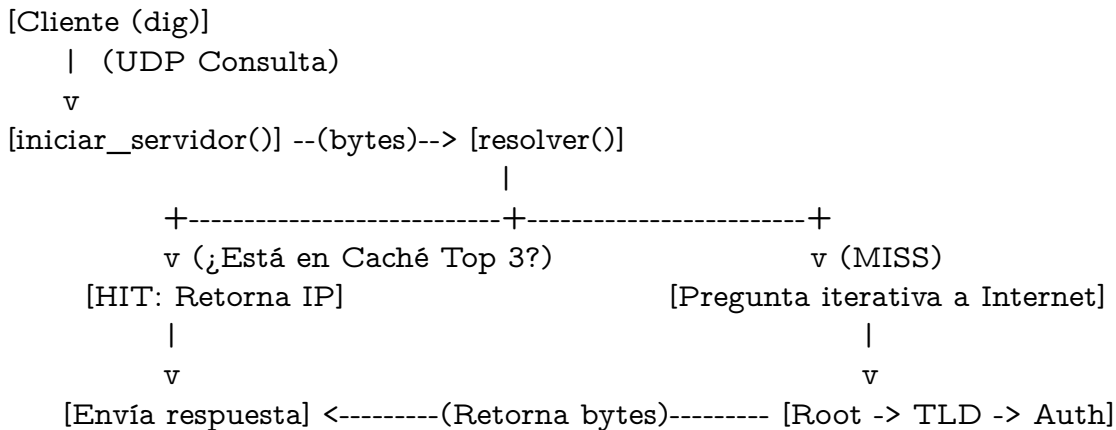


Actividad DNS: Construyamos un resolver

Integrantes: Francisca López
Rakhel Gomez Montero
Profesor: Ivana Bachmann
Auxiliar: Julián Ferreira
Fecha de entrega: 07 de septiembre de 2026
Santiago de Chile

1. Flujo de Funcionamiento y Ejecución del Código

El código se organiza en dos partes principales dentro de **resolver.py**: la recepción en red y la resolución lógica iterativa. El flujo básico de comunicación cuando llega una consulta de un cliente es el siguiente:



1.1. Cómo ejecutar el código

Para probar el resolver, se deben utilizar dos terminales en la máquina virtual:

1. **Terminal 1 (Servidor):** Iniciar el resolver ejecutando:

```
1 python3 resolver.py
```

El programa quedará escuchando consultas UDP en la dirección local 127.0.0.1 a través del puerto 8000.

2. **Terminal 2 (Cliente):** Realizar consultas usando **dig** apuntando al puerto 8000:

```
1 dig -p 8000 @127.0.0.1 eol.uchile.cl
```

2. Decisiones de Diseño

- Usamos un socket UDP (no orientado a conexión) bajo la familia **AF_INET**. De acuerdo al estándar de DNS (RFC 1035), las consultas comunes usan UDP porque es mucho más rápido al no requerir establecer una conexión previa (handshake). Si un paquete se pierde, simplemente el cliente asume un timeout y lo vuelve a intentar.

- Se configuró el puerto 8000 para el servidor local. Los puertos del 0 al 1023 son restringidos y requieren permisos de administrador (**sudo**), por lo que usar el 8000 facilita la ejecución y pruebas directo en el espacio de usuario.
- Para cumplir con guardar los 3 dominios más repetidos de las últimas 20 consultas recibidas, implementamos tres componentes:
 1. **Historial:** Una lista normal de Python que guarda los últimos 20 dominios consultados. Si se supera el límite de 20, sacamos el más antiguo usando `historial.pop(0)`.
 2. **Cálculo de favoritos (Top 3):** Usamos la librería nativa `collections.Counter` para contar los dominios del historial de forma eficiente y obtener el top 3 con el método `.most_common(3)`.
 3. **Almacenamiento de IPs:** Un diccionario que asocia los dominios con sus IPs resueltas (`{dominio: IP}`).
- Usamos un flag interno `is_client_query` para evitar que las consultas recursivas de infraestructura (como buscar la IP de un Name Server intermedio) contaminen el historial de consultas más repetidas del cliente.

3. Resultados y Respuestas a los Experimentos

3.1. Experimento 1: Resolución de `www.webofscience.com`

Al consultar por este sitio el servidor entra en un ciclo infinito de consultas recursivas intentando resolver el dominio, lo que causa un timeout en el cliente `dig`. Esto se debe a que el dominio tiene una dependencia circular de Name Servers sin registros *glue* (Glue Records) en su sección adicional. Para saber la IP de su Name Server se requiere consultar el mismo Name Server que estamos intentando resolver, y como resultado el resolver se consulta a sí mismo infinitamente. Se podría arreglar implementando un límite de recursión global (`max_depth = 10`) y un conjunto (`set()`) para registrar los Name Servers que se están consultando activamente, interrumpiendo el flujo de inmediato si detectamos que un dominio se repite en la pila.

3.2. Experimento 2: Resolución de `www.cc4303.bachmann.cl`

Para este sitio, el resolver da *timeout*, pero si se consulta en uno público (1.1.1.1) responde de inmediato con un error de tipo `status: NXDOMAIN` (el dominio no existe). Este suceso ocurre debido a que el dominio raíz existe pero el subdominio `www` no está configurado en sus registros. El resolver público recibe esta respuesta negativa y se la pasa de vuelta a

dig. Nuestro código básico, al estar diseñado para ignorar respuestas que no sean de tipo A o delegaciones, simplemente no devuelve nada (retorna `b""`), dejando al cliente colgado esperando hasta que expira por timeout.

3.3. Experimento 3: Redundancia y Round Robin

Al repetir consultas con el caché apagado, el orden de los Name Servers intermedios consultados y las IPs de respuesta varían en cada intento. Esto pasa porque DNS usa **Round Robin** para balancear la carga de consultas, rotando los servidores de su lista para que el tráfico de Internet se distribuya de forma pareja en la infraestructura de red.

3.4. Limitaciones de ignorar respuestas no estándar

1. **Incompatibilidad con CNAMEs externos:** Si el dominio es un alias que apunta a otra zona, el código ignorará el registro de alias y no podrá seguir la cadena para resolver la dirección IP real.
2. **Ausencia de manejo de errores formal:** En lugar de responder limpiamente con cabeceras de error (como `NXDOMAIN` o `SERVFAIL`), el resolver guarda silencio, haciendo que el usuario tenga que esperar innecesariamente un timeout.

4. Declaración de Uso de Inteligencia Artificial

Se declara el uso de herramientas de IA en esta actividad, siendo las estudiantes completamente responsables de lo entregado:

- **Herramienta utilizada:** Gemini 3.1 Pro.
- **Apoyo en el código:** Co-diagnóstico para solucionar el bug de comparación del caché usando `.values()` en el diccionario de dominios favoritos.
- **Apoyo en el informe:** Asistencia para estructurar y redactar el texto de este informe.
- **Control del estudiante:** Todas las pruebas, edición del código e integración en la máquina virtual Debian 13 fueron realizadas, revisadas y ejecutadas manualmente por las estudiantes.